

1) Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.
Digital Object Identifier 10.1109/ACCESS.2017.Doi Number

Optimizing XGBoost Hyperparameters for Credit Scoring Classification using Weighted Cognitive Avoidance Particle Swarm

Atul Vikas Lakra^{1,2}, Sudarson Jena², and Kaushik Mishra³

¹Veer Surendra Sai University of Technology, Burla 768018, Odisha, India

²Sambalpur University Institute of Information Technology, Sambalpur University, Burla 768016, Odisha, India

³Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education (MAHE), Manipal 576104, Karnataka, India

Corresponding authors: Sudarson Jena (e-mail: sjena@suit.ac.in) and Kaushik Mishra (e-mail: kaushik.mishra@manipal.edu)

ABSTRACT Decision trees in machine learning achieved satisfactory performance in classification. Decision trees offer the advantage of handling high-dimensional and complexly correlated data through feature combination and selection. Extreme Gradient Boosting (XGBoost) overcomes the issue of overfitting in decision trees by integrating multiple tree models. Weighted cognitive avoidance particle swarm optimization for the XGBoost (WCAPSO-XGB) model has been proposed for credit score classification. The structure of the XGBoost (XGB) model is determined by the hyperparameters, which must be initialized prior to model evaluation. The optimal hyperparameter values for the XGBoost model can vary significantly depending on the specific problem at hand. Manually setting these hyperparameters can be a time-consuming process. Therefore, to automate this process, weighted cognitive avoidance particle swarm optimization (WCAPSO) is employed for hyperparameter optimization. The novelty of the proposed work includes a modified version of the particle swarm optimizer (PSO), which is used to address the issue of getting trapped into a local optimal solution. The proposed WCAPSO restricts the movement of particles toward local optima and enhances exploration of the search space for better solutions. Here, the hyperparameters of XGBoost are represented as particles in WCAPSO, which defines the state and its movement in a continuous search space. The proposed WCAPSO-XGB model tunes the hyperparameters of XGBoost and classifies the credit scoring, and the experimental results are compared with various classifier such as Random Forest (RF), K-neighbors (KNN), Gaussian Naive Bayes (NB), AdaBoost, Gradient Boosting, Logistic Regression (LR), Neural Network (NN), Decision Tree (DT) and Linear Discriminant Analysis (LDA), and hyperparameter optimization methods, such as Grid Search (GS), Random Search (RS), Bays Optimization, Optuna Optimization, Hybrid Snake Optimizer Algorithm (HSOA), Exploratory Cuckoo Search, island Cuckoo Search (iCSPM and iCSPM2), and Improved SSA (ISSA) with HDPM, on four different datasets with a varying number of instances from small to large. The experimental results and analysis show that the proposed model secures better results in accuracy of 76.75%, 82.43%, 88.98%, and 88.74%; F1-score of 66.52%, 89.44%, 93.53%, and 90.64%; precision of 78.79%, 84.31%, 88.46%, and 90.41%; and recall of 64.07%, 95.24%, 99.21%, and 90.87% in the UCI German, UCI Taiwan dataset, P2P US lending club, and Credit scoring Kaggle dataset, respectively. Further, the overall confidence interval of the proposed model ranges from 82% to 83% in accuracy, 73.1% to 74.2% in F1-score, and 71% to 72% in AUC, with a confidence of 95% measured over 20 evaluations on each dataset.

INDEX TERMS Credit score, ensemble method, hyperparameter optimization, particle swarm optimization (PSO), XGboost

II. INTRODUCTION

In today's scenarios, everyone has personal assets, such as a plot, house, car, etc., which have increased drastically after the Covid-19 pandemic. Acquiring these assets for self or family depends upon personal financial background and source of income, based on which various financial companies or banks provide loans to the customers. Banks or financial companies check the substantial profits and significant financial risks before granting such loans by carefully analyzing the applicant's basic information and credit history comprehensively. Granting loans to potential borrowers is a critical operation for lending institutions worldwide.

Machine learning (ML) techniques, such as neural networks (NN), support vector machines (SVM), and decision trees (DT), have gained attention for credit scoring. DT models [1], favored for their interpretability, utilize tree decision structures to capture interactive information between credit features and identify important variables. Authors [2] established a Bayesian latent variable model based on a classification regression tree for bank credit data, achieving higher prediction accuracy and satisfactory results. The authors of [3] introduced a novel model based on an advanced tree-based approach that outperformed benchmark models on public credit datasets. DT's ability to handle complex financial data and provide intuitive interpretations makes it a desirable choice. Ensemble methods, which combine multiple classifiers, have been extensively applied to credit scoring. [4] employed class-wise bagging, while authors [5] proposed ensemble classification based on supervised clustering, both of which achieved higher prediction accuracy compared to single classifiers. The authors [3, 6] combined bagging with stacking in an ensemble credit model and demonstrated improved accuracy, recall, precision, ROC-AUC, and F1 scores. Ensemble learning demonstrated satisfactory performance in credit scoring due to its ability to combine multiple classifiers and learn from different parts of the data and feature space. SVM, which is suitable for binary classification in credit scoring, aims to identify a hyperplane that correctly separates training instances into correct classes. The authors [7] used SVM along with the F-score for feature selection, resulting in superior precision and calculation time. Authors [8] proposed a kernel-free SVM model that avoids kernel parameter selection, yielding promising results on personal credit data. However, SVM boundaries can be biased toward minority instances in imbalanced datasets, which reduces prediction accuracy. SVM also lacks explanatory power for lending decision-making. In NN models [9], inspired by the brain's synaptic connections, weights are iteratively adjusted to minimize prediction errors, making them effective in identifying patterns in complex financial credit data. Chuang and Huang proposed a hybrid credit scoring model based on NN, achieving more accurate results and preventing non-potential customer churn. Fonseca introduced a two-stage process that combines fuzzy inference

models with NN, and it demonstrated satisfactory predictability. However, NN models lack explanation capabilities, are time-consuming, and are prone to overfitting.

Extreme Gradient Boosting (XGBoost) [10, 11], an advanced gradient boosting algorithm, has shown promising results in data competitions and bankruptcy prediction. It handles imbalanced data and allows custom objective functions and evaluation indicators for special situations. XGBoost's interpretability helps explain feature contributions and relationships. Authors [1, 12, 13] proposed XGBoost-based credit scoring models with cluster-based undersampling and cost-sensitive approaches, achieving superior accuracy and AUC compared to other models. Although XGBoost performed well, the hyperparameter settings significantly affect the model performance. Manual tuning is time-consuming and resource-intensive; thus, hyperparameter optimization methods such as grid search (GS), random search (RS) [14], tree-structured Parzen estimator (TPE) [12], and particle swarm optimization (PSO). PSO is a computational intelligence technique that performs hyperparameter optimization in a continuous search space. Author [17] proposed an ensemble model that combines AdaBoost with PSO to optimize neural network weights, and achieved superior effectiveness.

The PSO algorithm is a metaheuristic algorithm, and some similar algorithms have been developed by researchers. Such as the hybrid snake optimization algorithm (HSOA), which particularly combines oppositional-mutual learning (OML) with the initialization step of the SO algorithm and dynamic polynomial mutation (DPM) with both the initialization and optimization processes [34-37]. The main intention of these hybrid methods is to enhance the population diversity in the SO framework and to enhance its searchability during the optimization process. In addition to HSOA, other hybridizations using snake algorithms have been attempted. For example, the integration of snake models with Particle Swarm Optimization (S-PSO) [38] has been used successfully in fields like medical image processing and sunspot tracking, taking advantage of the high-level object detection abilities of snake models and the optimization capabilities of PSO.

The Cuckoo Search (CS) algorithm [39, 40, 41] is a metaheuristic optimization algorithm based on the brood parasitism behaviour of some cuckoo species and the Levy flight behaviour of some birds and fruit flies. Its performance may be very sensitive to parameter selection, such as the population size and the nest abandonment rate. For large-dimensional problems, the algorithm requires a significant number of iterations to converge to a good solution. A variant of Cuckoo Search, the island Cuckoo Search (iCSPM) [42] method solves optimization issues by using the island model and an extremely disruptive polynomial mutation. The Island Cuckoo Search with Highly Disruptive Polynomial Mutation

(iCSPM) algorithm was improved with iCSPM2 [42]. Compared to the original iCSPM, it performs better on optimization issues thanks to some improvements, such as the use of various mutation techniques and elite opposition-based learning. Both algorithms, like many metaheuristics, can be sensitive to parameter settings; in some situations, they may prematurely converge or become trapped in local optima.

The Salp Swarm Algorithm (SSA) [43] is a single-objective optimization algorithm that draws inspiration from the swarming behaviour of salps, which are marine invertebrates that chain themselves and swim in groups. However, the original SSA tends to experience premature convergence and is not efficient when applied to highly dimensional optimization problems. To address these issues, refined SSA variants have been proposed, and they tend to adopt methods such as Highly Disruptive Polynomial Mutation (HDPM) [44]. SSA is susceptible to premature convergence, where it can be trapped in local optima prematurely in the search process because of the weakness of its optimization operators. In addition, it cannot do a good job with high-dimensional optimization problems. The combination of SSA with other algorithms, e.g., Particle Swarm Optimization (HSSAPSO) [45], also reflects attempts at combining the strengths of various metaheuristic strategies.

The continuous nature of PSO and many metaheuristic algorithms uses PSO to enhance the strength of existing algorithms. Therefore, standard PSO is suitable for XGBoost hyperparameter optimization. However, standard PSO can fall into local optima. The functioning of PSO is highly dependent on its parameter settings, including the population size and weighting factors in the update equations. In addition, according to the “No Free Lunch Theorem for optimization” proposed by Wolpert and Macready [46], no particular algorithm can exhibit better results for all types of problems due to the stochastic nature of optimization algorithms. This is the main cause of the development of new kinds of algorithms proposed for different kinds of problems. In this direction, this paper addresses the development of a new variant of PSO using the principle of weighted cognitive avoidance in the standard particle swarm optimization, named weighted cognitive avoidance particle swarm optimization (WCAPSO). The proposed algorithm guides particle swarm movement to enhance diversity and avoid local optima.

The major contributions of the paper are as follows:

1. A modified PSO algorithm named as WCAPSO is proposed for the XGBoost model.
2. The above optimized model is used for credit scoring.
3. This study mainly focused on hyperparameter optimization of the XGBoost model and verifying its performance.
4. The effectiveness of the proposed model is verified by using four standard datasets, such as UCI

German, UCI Taiwan, P2P US Landing Club, and Credit Score Kaggle.

The rest of the paper is organized as follows: Section 2 presents the related work; Section 3 presents the methods and materials used in the paper; Section 4 presents the results and discussion; and Section 5 concludes the paper and previews its future works.

III. RELATED WORK

Most of the credit scoring research work aims to enhance the classification performance in terms of accuracy. There are two major categories of credit scoring methods. One is that statistical methods predict loan defaulters using a suitable sequential arrangement of input variables. A commonly used statistical model, logistic regression (LR), is used in credit scoring due to its simplicity of implementation [15]. LR utilizes the maximum likelihood function and gradient descent to classify parameters as default or non-default. However, LR works better with linear input variables and limits its performance with complex nonlinear credit score datasets.

In the second category, machine learning (ML) models have better classification capabilities when handling complex nonlinear data [16]. Many research works have shown that ML models outperform statistical methods in terms of prediction accuracy for financial data. Authors [18] proposed an NN for a hybrid genetic algorithm, achieving better classification than the original genetic algorithm. Author [40] proposed a cuckoo search-based NN model for predicting financial distress. The authors [19] introduced SVM with a profit-driven feature, which simultaneously constructs classifiers and selects features, and performed better in credit scoring for a Chilean bank. In [20], the authors applied DT and LR to Lending Club data and obtained satisfactory default prediction performance. Using a single classifier with limited training data and a small hypothesis space can result in overfitting and limitations when capturing complex patterns. Ensemble learning is a promising approach to address the limitations of single classifiers in credit scoring. Single classifiers often suffer from noise and redundant attributes, which results in unstable model performance when handling complex data.

Bagging and boosting are two ensemble learning methods. Bagging involves generating random subsets via repeated random sampling and obtaining the final result by aggregating the predictions of models trained on each subset. Random Forest (RF) is a well-known bagging classifier that trains multiple trees on random subsets and combines their results. Bagging-based approaches are less affected by noise than single classifiers. Bagging-DT and Bagging-NN strategies have been introduced to improve the performance of single classifiers.

Boosting, on the other hand, builds models sequentially by combining weak learners based on their weights. AdaBoost is a popular algorithm that assigns more weights to weak classifiers with low error rates and decreases weights for high error rates in each iteration during its evaluation. It can use various base learners, with DT being the most common choice. AdaBoost-NN [21], which employs a multilayer perceptron as the base classifier, has also demonstrated satisfactory results in credit scoring. AdaBoost's simplicity helps prevent overfitting; however, it may experience reduced classification accuracy due to imbalanced data.

The Gradient Boosting Decision Tree (GBDT) [1, 22] employs regression trees as weak classifiers and adds a new tree in each iteration to fit the residuals. GBDT exhibits flexibility in handling various types of credit data and performs well in handling exceptional or fraudulent applications. XGBoost, which is an extension of GBDT, incorporates first and second derivatives, resulting in a more accurate loss function. It can split trees more precisely, making them more suitable for financial data characteristics. XGBoost includes a regularization term to control model complexity and prevent overfitting, especially when dealing with small sample credit data. In addition, XGBoost has mechanisms to handle missing values by learning the direction of credit sample segmentation. These advantages contribute to its reasonable predictive performance in credit scoring.

Hyperparameter optimization is crucial for fully exploring the effectiveness of XGBoost. Grid search (GS) exhaustively tests all combinations of candidate parameters, whereas random search (RS) randomly selects sample points for evaluation, making it faster than GS. However, due to the relatively large number of hyperparameters in XGBoost, these methods may not be feasible. XGBoost was optimized [12] by the tree-structured Parzen estimator (TPE), which outperformed baseline models in terms of accuracy, error rate, AUC, and Brier score. However, TPE requires independent hyperparameters, and highly correlated hyperparameters cannot be optimized simultaneously. Particle swarm optimization (PSO) overcomes these limitations by cooperatively searching for the optimal value based on information sharing among individuals in the swarm. PSO is suitable for continuous optimization problems and is not restricted by objective function constraints. It also allows for hyperparameter dependencies.

Considering XGBoost's [11, 23] strengths in handling missing values, accurate loss function solutions, overfitting prevention, and PSO's advantages in optimizing parameters in high-dimensional continuous space, an XGBoost model optimized by improved adaptive particle swarm optimization is proposed for high-precision credit scoring. The objective function is the fitness of the particle, thereby allowing simultaneous optimization of multiple hyperparameters in

continuous space. This approach allows XGBoost to perform optimally, mitigate the impact of credit data imbalance, determine reasonable splitting directions for missing values, and enhance overall prediction accuracy.

IV. Methodology

A. XGBoost

Extreme gradient boosting creates decision trees in a sequential form. In this algorithm, weights are assigned to all independent variables. These weighted independent variables were passed into the decision tree to predict the results. Variables that lead to wrong prediction weights are increased and then passed into other decision trees for prediction. In consideration that a weak decision tree is better than a random decision tree prediction. This helps in getting an idea about feature importance and parameter tuning. Using information from a weak decision tree, a strong decision tree can be built and can reduce error in classification. Decision trees are built one after another to reduce classification error through the number of iterations. In [10, 23, 25], the authors use boosting to combine the number of decision trees to enhance the performance of the XGBoost model. XGBoost can be used for various types of machine learning, including classification, regression, and ranking.

1) XGBoost Hyperparameter

A hyperparameter of a model refers to a parameter that be initialized prior to model evaluation. The hyperparameter also defines the structure or architecture of the model. Hyperparameter tuning for machine learning algorithm is difficult and time-consuming task. The hyperparameter tuning depends on a deep understanding of the model and the role of each hyperparameter in the model. The tuning also varies from one problem domain to another. Thus, tuning the hyperparameter correctly requires research and experience in tuning hyperparameters for the model. To simplify the task, hyperparameter optimization is critical to fine-tuning the hyperparameters of any model in any problem domain. In this paper, the proposed weighted cognitive avoidance particle swarm optimization (WCAPSO) was used as hyperparameter optimization for the XGBoost model. In the XGBoost model, hyperparameters [26] control the number of decision trees, depth of tree, and many others, which are described as follows:

- **learning_rate:** Describe the weight value in each iteration and how it is used to avoid overfitting. The range is 0.05 to 0.2 in order to enhance the quality of learning and reduce the risk of overfitting.
- **max_depth:** Describe the maximum depth of the decision tree. The range is 3 to 10 for dealing with simple to complex trees with various splits for capturing and learning various patterns during training. The XGBoost model, beyond the above range, it might underfit or overfit on complex and simple datasets, respectively.
- **n_estimators:** Number of trees used to build strong learner decision trees. The range is 10 to 100 based on the

experimental results. In general, a higher value is suitable for enhancing performance when the learning rate is negligible.

- **min_child_weight:** Describe the sum of the minimum leaf node sample weights. The range is 1 to 7, which controls the overfitting of the model.
- **gamma:** Describe the requirement of node partitioning using minimum loss function reduction. The range is set from 0 to 0.4 to facilitate further partitioning on the node during tree construction without loss of important information.
- **colsample_bytree:** Number of features used in percentage per tree. The range is set from 0.3 to 0.9 to ensure learning from different subsets of features of the dataset.
- **subsample:** Describe the percentage of sample data selected at each boosting iteration. The range is 0.5 to 1 to maintain randomness and reduce overfitting.

Optimization of hyperparameters of the XGBoost model using WCAPSO: each particle in the population represents a candidate solution. A particle or candidate solution is a set of hyperparameters and its value are given above. The particle can be represented as follows:

Particle (P_i) = { learning_rate ($H1_i$),
max_depth ($H2_i$),
n_estimators ($H3_i$),
min_child_weight ($H4_i$),
min_child_weight ($H4_i$),
gamma ($H5_i$),
colsample_bytree ($H5_i$),
colsample_bytree ($H6_i$),
subsample ($H7_i$) }

$P_i = \{ H1_i, H2_i, H3_i, H4_i, H5_i, H6_i, H7_i \}$

Here, P_i represents the i^{th} particle in the swarm, and $H1_i$ to $H7_i$ represent the seven hyperparameters considered in this paper. Each candidate solution (particle) is used to build an XGBoost model, and the model performance is evaluated in terms of accuracy measured using a fitness function defined in WCAPSO. Further, after each iteration in the WCAPSO, $H1$ to $H7$ values of the candidate solution are updated in order to maximize the fitness function (discussed in the next section).

B. Proposed weighted cognitive avoidance particle swarm optimization

Particle swarm optimization (PSO) [27] is inspired by the movement of birds searching for food in the real world. PSO describes each bird's velocity, which describes the speed of the bird's movement, and position, which describes the direction in which the birds move. In PSO [28], particles represent birds and movements with the velocity of the particles in the n-dimensional search space. Each particle obtains the optimal solution and stores it as a personal best (PB) solution. Through the interaction among the particles, the

global best (GB) solution is derived. The velocity and position of each particle were modified using PB and GB at each iteration. The importance of each particle is determined by its fitness function, which also determines the power to influence other particles in the population. The fitness function used to obtain the optimal solution as PB and GB for each particle. In PSO, the i^{th} particle at the t^{th} iteration in n-dimension, the position (P), velocity (V), personal best (PB), and global best (GB) are represented as follows.

$$\begin{aligned} P(t) &= p_{i1}, p_{i2}, p_{i3}, \dots, p_{in} \\ V_i(t) &= v_{i1}, v_{i2}, v_{i3}, \dots, v_{in} \\ PB_i(t) &= PB_{i1}, PB_{i2}, PB_{i3}, \dots, PB_{in} \\ GB_i(t) &= GB_{i1}, GB_{i2}, GB_{i3}, \dots, GB_{in} \end{aligned}$$

$$\begin{aligned} V_{ij}(t+1) &= V_{ij}(t) + C_1 \times R_1 \times (PB_{ij}(t) - P_{ij}(t)) \\ &\quad + C_2 \times R_2 \times (GB_{ij}(t) - P_{ij}(t)) \end{aligned} \quad (1)$$

$$P_{ij}(t+1) = P_{ij}(t) + V_{ij}(t+1) \quad (2)$$

The position and velocity of each individual particle are updated at time $t+1$ using equations (1) and (2). Here, $R1$ and $R2$ are randomly generated uniformly distributed numbers in the range between 0 and 1. $C1$ and $C2$ are constants, whose values range from 0 to 2. $C1$ is the acceleration coefficient, also referred to as cognitive acceleration, which directs each particle's movement in the direction of the particle's personal best (PB) optimal solution, and $C2$ is the acceleration coefficient, also referred to as social acceleration, which directs each particle in the direction of the global best (GB) optimal solution. In equation 1, the term $C1 \times R1 \times (Pb_{ij}(t) - P_{ij}(t))$ determines the particle's ability to obtain its personal best optimal solution, and the term $C2 \times R2 \times (Gb_{ij}(t) - P_{ij}(t))$ determines how each particle collaborates with other particles in the population. Acceleration coefficients are used to improve the pace of change in velocity. To further improve the performance of the PSO algorithm, the particle's velocity at time t in (1). is multiplied by the inertia weight (ω) given in (3) to change the acceleration rather than a fixed enhancement in velocity at each iteration of the algorithm.

$$\begin{aligned} V_{ij}(t+1) &= \omega \times V_{ij}(t) + C_1 \times R_1 \times (PB_{ij}(t) - P_{ij}(t)) \\ &\quad + C_2 \times R_2 \\ &\quad \times (GB_{ij}(t) - P_{ij}(t)) \end{aligned} \quad (3)$$

PSO-RANDIW [21] uses a randomly generated random inertia weight in (4) has been proposed by the author instead of using a fixed value of (ω) in each iteration.

$$\omega = 0.5 + \frac{1}{2} \times rand() \quad (4)$$

The PSO algorithm (Algorithm 1) initializes the inertia weight (ω), position (Pos), velocity (Vel), and swarm of particle P_i , randomly within a specific search space range. The inertia weight (ω) was used to multiply with the current velocity

$V_{ij}(t)$ during the evaluation of $V_{ij}(t + 1)$ (4). The fitness value of each particle was evaluated using the fitness function (objective function). Based on the fitness value of the particle PB for each particle, the best PB was assigned to the GB value.

Algorithm 1: PSO algorithm

```

1 Initialize Position (Pos), Velocity (Vel), Iteration, Inertia
  weights ( $\omega$ ) and Search Space(S)
2 Generate random particles (P) in the search space
3 for i in particle (P):
    Evaluate fitness function  $f_i(\text{Pos}_i)$ 
    Update PB and GB
end for
for it in range(iteration):
    for i in particle (P):
        update Veli and Posi [using (2) and (3)]
        if Veli > Vmax: Veli = Vmax
        elseif Veli < Vmin: Veli = Vmin
        for s in S:
            if Posi[s] > Posmax: Posi[s] = Posmax
            elseif Posi[s] < Posmin: Posi[s] = Posmin
            Update PB and GB
        end for
        calculate Fitness function  $f_i(\text{Pos}_i)$ 
        Update PB and GB
    end for
end for

```

These PB and GB values represent the particular particle position in the search space (S). The velocity of the particle is calculated using PB and GB along with its current position (3). After the calculation of the velocity, if its value is within the range, then the velocity is added to the current position of the particle to evaluate the new position of the particle (2). In the next iteration, repeat the steps to evaluate the fitness and update the PB and GB, and update the velocity and position. The iteration will continue for 'n' number of times or until the desired result has been achieved.

In PSO, particles converge towards the optimal solution with the help of PB and GB after each remaining iteration (Algorithm 1). The movement of particles towards the global best solution may end up at the local optimal solution in the search space instead of the global optimal solution for the problem. So, it can be possible to obtain the optimal solution using PB, but due to the interaction of other particles and GB, particles may move in the wrong direction and accelerate. To avoid such a scenario, another term is added called cognitive avoidance [28], [29] in the velocity update equation (3). So, the new velocity equation (5) helps the particle avoid taking the wrong direction and enhances the exploration for the randomly initialized particle population.

$$\begin{aligned}
 V_{ij}(t + 1) = & \omega \times V_{ij}(t) + C_1 \times R_1 \times (PB_{ij}(t) - P_{ij}(t)) \\
 & + C_2 \times R_2 \times (GB_{ij}(t) - P_{ij}(t)) \\
 & - C_3 \times R_3 \\
 & \times (PW_{ij}(t) - P_{ij}(t))
 \end{aligned} \quad (5)$$

Here in (5), PW denotes the particle's personal worst position (PW). The PW of an i^{th} particle is expressed as $PW_i(t) = (PW_{i1}, PW_{i2}, PW_{i3}, \dots, PW_{in})$, and C_3 is a constant value ranging from 0 to 2. The personal worst position helps to determine the right movement of the particle along with the influence of the global best and local best positions for finding the global optimal solution of the problem. The cognitive avoidance term helps to avoid getting trapped in the local optimal solution and prevents particles from taking the wrong movement and direction.

In the above (5), it has been observed through experiment that the acceleration in particle velocity has been reduced. This reduction in acceleration leads to the PSO convergence and exploration rate. Hence, this paper proposes weighted cognitive avoidance by multiplying ωc in the cognitive avoidance term. The updated equation (5) is as follows.

$$\begin{aligned}
 V_{ij}(t + 1) = & \omega_{ij} \times V_{ij}(t) + C_1 \times R_1 \times (PB_{ij}(t) - P_{ij}(t)) \\
 & + C_2 \times R_2 \times (GB_{ij}(t) - P_{ij}(t)) \\
 & - \omega c_{ij}(t) \times R_3 \\
 & \times (PW_{ij}(t) - P_{ij}(t))
 \end{aligned} \quad (6)$$

$$\omega c_{ij}(t + 1) = \omega c_{ij}(t) + [((PB_{ij}(t) - GB_{ij}(t)) / GB_{ij}(t))] \times L \quad (7)$$

where, ωc_{ij} is the weight for cognitive avoidance in the i^{th} iteration at the j^{th} dimension and ω_{ij} is random inertia weight for velocity in the i^{th} iteration at the j^{th} dimension, which is fixed for all dimensions in equation (5). In equation (7), L denotes the learning rate ranging from 0.1 and 0.9. Initially, the cognitive avoidance weight (ωc) (7) randomly generated between the values 0.4 and 0.9, and during each iteration (ωc) updated using equation 7. The proposed weighted cognitive avoidance helps the PSO algorithm to enhance overall performance and to enhance convergence and exploration rate. The proposed weighted cognitive avoidance particle swarm optimization algorithm is described as follows in Algorithm 2

Algorithm 2: Weighted cognitive avoidance particle swarm optimization (WCAPSO)

```

Initialize Position (Pos), Velocity (Vel), Iteration, Inertia
weights ( $\omega$ ,  $\omega c$ ) and Hyperparameter Space(S)
Generate particles (P) randomly from the
hyperparameter space for the XGBoost model.
for i in particle (P):
    Evaluate fitness function  $f_i(\text{Pos}_i)$ 
    Update PB, GB and PW

```

```

end for
for it in range(iteration):
    for i in particle (P):
        Update Veli and Posi
        if Veli > Vmax: Veli = Vmax
        elseif Veli < Vmin: Veli = Vmin
        for s in S:
            if Posi[s] > Posmax: Posi[s] = Posmax
            elseif Posi[s] < Posmin: Posi[s] = Posmin
            Update  $\omega c$  (PB, GB, L)
        end for
        calculate Fitness function  $f_i$  (Posi)
        Update PB, GB and PW
    end for
    calculate Fitness function  $f_i$  (Posi)
    Update PB, GB and PW
end for

```

C. WCAPSO for the XGBoost model.

The proposed WCAPSO algorithm is used as a hyperparameter optimizer for the XGBoost model WCAPSO-XGB. In this paper, various credit score datasets were used to classify credit risk as good or bad. The complete process of the proposed WCAPSO-XGBoost model workflow is shown in Fig. 1 and includes training, testing, and obtaining optimized hyperparameters of the XGBoost model for credit score datasets. Initially, the hyperparameters of the XGBoost model, such as learning_rate, max_depth, n_estimators, min_child_weight, gamma, colsample_bytree, and subsample, are the optimization target, and the search space is based on these hyperparameters. Initially, random populations of particles are generated from the hyperparameter space, and velocity, PB, and PW are set to zero. In each iteration, the fitness value of particles was evaluated using the XGBoost model. When the current evaluated fitness value is greater than the previous PB value, then update PB with the current fitness value; else update PW with the current fitness value. Among the particles, the particle with the highest PB value will be assigned as GB. Using equation (6), the particle's new velocity was evaluated using current velocity, PB, and PW. The weight for cognitive avoidance (ωc) of each particle are evaluated using equation (7). At the end of each iteration, the position of particles is evaluate using equation (2).

In each iteration of WCAPSO, a new set of hyperparameters is evaluated as the particle's position and passed to the XGBoost model until the process obtains a new, higher fitness value compared to the previous fitness value. Once the termination criteria are met, WCAPSO returns the optimized hyperparameter. Using the optimized hyperparameter of the XGBoost model, the final model is constructed, and performance matrices are evaluated on the test dataset.

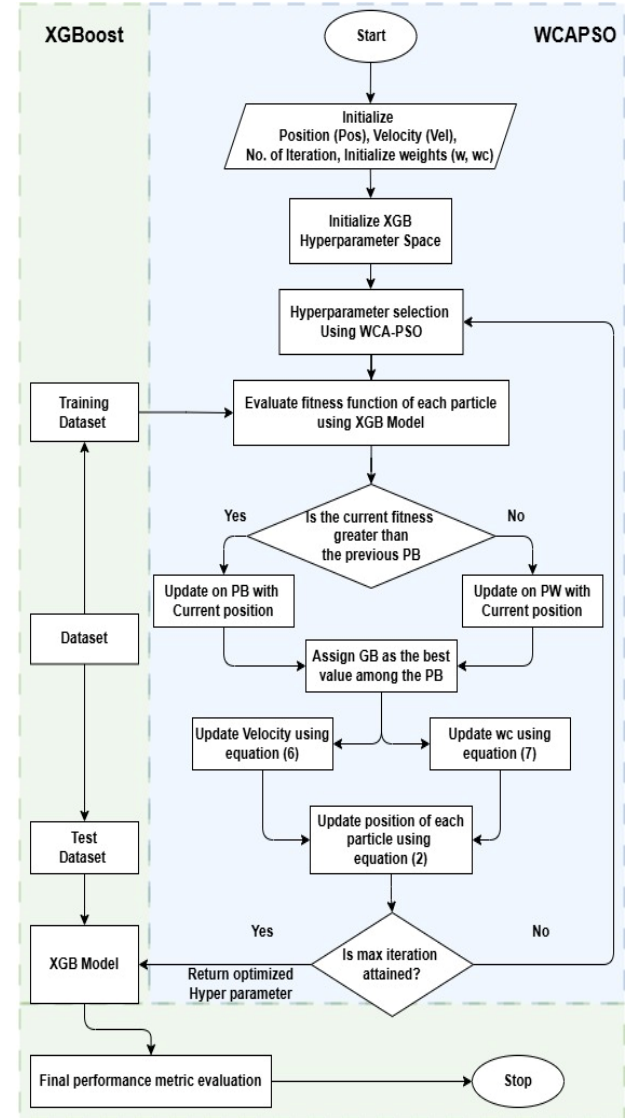


Figure 1. WCAPSO-XGBoost proposed model work flow

V. RESULTS AND DISCUSSION

A. Dataset

The performance of the proposed and traditional methods is evaluated using the following credit score datasets listed in Table I. For training and testing, the datasets are divided in a 70:30 ratio, and 10-fold cross-validation is used to overcome overfitting and underfitting of the models.

Using feature selection, the required features are selected, and unwanted data, such as missing values, imbalanced data, and less correlated data, are removed. This helps construct an efficient and simpler model, reduces dimensionality problems, and speeds up model learning with less complexity in evaluation.

TABLE I
CREDIT SCORE DATASET

Dataset	# Instances	# Attributes
UCI Germany	1000	20
UCI Taiwan	30000	24
Credit Score Kaggle	10000	28
P2P US Lending Club	396030	27

B. Evaluation Criteria

In this paper, for credit score classification as good or bad, accuracy (ACC) is one of the most important criteria to evaluate the model's performance. For a more detailed evaluation, precision and recall measures are considered to differentiate good and bad scores for loan allocation to customers. Precision is high if the model correctly classifies a good score correctly, meaning the portion of positive samples in positive cases, and similarly, recall is high if the percentage of predicted positive cases is among the total number of positive cases. Another evaluation criterion called the F1-score checks the model's performance on an imbalanced dataset. The F1-score evaluates the mean precision and recall as an outcome (Table II). Along with the above measure, the criteria for selecting a better model for the classification of credit scores were evaluated using the ROC-AUC curve. The ROC curve (receiver operating characteristic curve) summarizes different confusion matrices at different threshold values. AUC (area under the curve) helps to identify the model with a high AUC as the better model to select because it can handle unbalanced datasets.

TABLE II
EVALUATION CRITERIA

Evaluation Criteria	Formula
Accuracy	$\frac{TP + TN}{TP + FP + TN + FN}$
Precision	$\frac{TP}{TP + FP}$
Recall	$\frac{TP}{TP + FN}$
F1 Score	$2 \times \frac{Precision * Recall}{Precision + Recall}$

C. Performance Evaluation

The experiment was conducted on Intel Xeon Scalable processors, 28 cores per processor, and 64 GB of memory, using Python. To evaluate and compare the performance of the proposed model, various classification algorithms are considered. Widely used classification algorithms such as Random Forest (RF), K-neighbors (KNN), Gaussian Naive Bayes (NB), AdaBoost, Gradient Boosting, Logistic Regression (LR), Neural Network (NN), decision tree (DT), and Linear Discriminant Analysis (LDA) are compared with the default XGB model with credit score datasets.

In the UCI German dataset [31] (Table III), XGB achieved accuracy (74%) along with the highest precision and F1-score (0.5229). It was also observed that the Gaussian Naive Bayes model achieved the highest recall (0.563), but the accuracy was not good compared to most of the models. Similarly, RF achieved the highest precision (0.6212) and accuracy (73%). In Fig. 2, the ROC-AUC curve shows the XGB model covers 66.5% of the area, indicating that the XGB model is a better model for the UCI German dataset in terms of all performance measures.

TABLE III
PERFORMANCE EVALUATION OF VARIOUS CLASSIFICATION ALGORITHMS ON THE UCI GERMAN AND UCI TAIWAN DATASETS

Dataset	UCI German				UCI Taiwan			
Algorithm	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
LR	73.75	0.5946	0.3697	0.456	78.24	0.7825	0.9999	0.8779
LDA	72.5	0.5556	0.3782	0.45	81.34	0.8241	0.9683	0.8904
KNN	67.75	0.4	0.1681	0.2367	75.37	0.8013	0.9111	0.8527
DT	64.25	0.3929	0.3697	0.381	72.58	0.8353	0.8091	0.822
NB	68.25	0.4718	0.563	0.5134	37.4	0.8682	0.2358	0.3709
RF	73	0.6212	0.3445	0.4432	81.6	0.843	0.9398	0.8888
SVM	70.75	0.5556	0.084	0.146	78.42	0.7851	0.997	0.8785
AdaBoost	73.75	0.5854	0.4034	0.4776	81.69	0.8322	0.9594	0.8913
Gradient Boosting	72.5	0.5652	0.3277	0.4149	81.49	0.8408	0.9419	0.8884
Neural Network	71.25	0.5175	0.4958	0.5064	77.63	0.8157	0.9227	0.8659
XGB	74	0.5758	0.479	0.5229	82.32	0.843	0.9511	0.8938

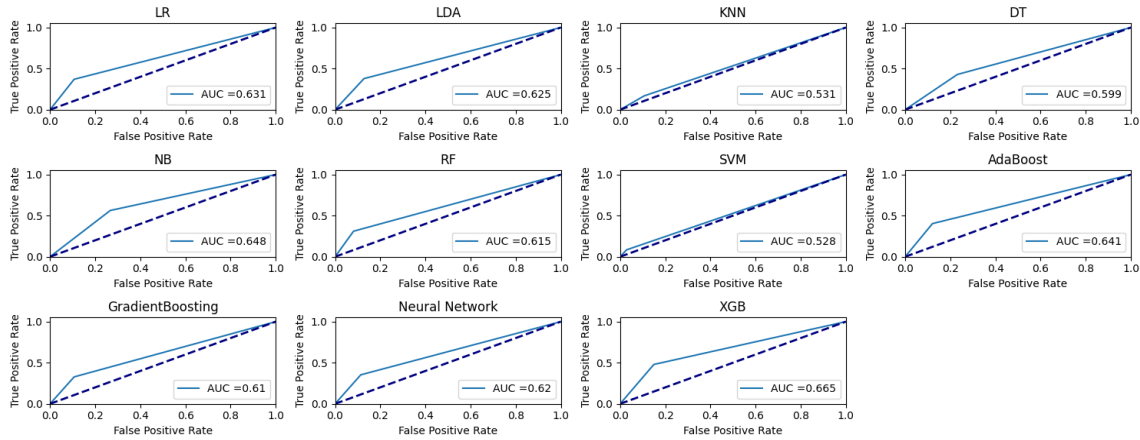


Figure 2. ROC-AUC Curve of Classification algorithm on UCI German dataset

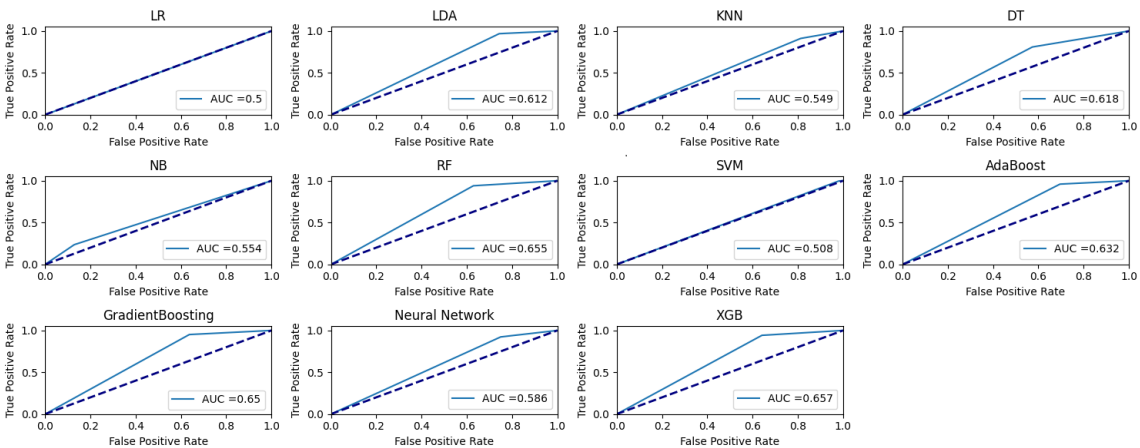


Figure 3. ROC-AUC Curve of Classification algorithm on UCI Taiwan dataset

In the UCI Taiwan dataset [30] (Table III), the XGB model achieved an accuracy (82.32%) along with the highest F1-score (0.8938). It is also observed that the LR (logistic regression) model achieved the highest recall (0.9999); this indicates the model has biased classification, and other performance criteria are not good compared to other models. The Gaussian Naive Bayes (NB) model achieved the highest precision (0.8682), but its accuracy was not good compared to any other model. In Fig. 3, the ROC-AUC curve shows the RF model covers 65.5%, whereas the XGB model covers 65.7%. Thus, the XGB model is a better model for the UCI Taiwan dataset in terms of handling unbalanced datasets and is the preferred model for classification.

In the Credit Scoring Kaggle dataset [32] (Table IV), the XGB model obtained the highest score in terms of accuracy (85.73%), precision (0.9275), and F1-score (0.8987). The random forest (RF) model performed 0.64% less accurately than the XGB model in terms of accuracy. In terms of precision, the XGB model performs 0.0256 points better than

the Gaussian Naive Bayes (NB) model. The random forest (RF) model achieved the highest precision (0.9026). In Fig. 4, the XGB and RF models achieved AUC-ROC scores of 0.83 and 0.824, respectively, indicating the capability of handling unbalanced datasets efficiently.

In the P2P US Lending Club dataset [33] (Table IV), the XGB model secures accuracy (88.92%) better than the Random Forest (RF) and LR (Logistic Regression) models, and both RF and LR models demonstrated the same performance in accuracy (88.9%). The Decision Tree (DT) model performed best in precision (0.8991), and Linear Discriminant Analysis (LDA) performs highest in recall (1). The XGB model obtained the highest in F1-score (0.9358), indicating it can handle unbalanced datasets efficiently compared to DT and LDA with high accuracy. In Fig. 5, the ROC-AUC curve shows that the XGB model covers the maximum area (0.736) in the curve compared to any other model.

TABLE IV

PERFORMANCE EVALUATION OF VARIOUS CLASSIFICATION ALGORITHMS ON CREDIT SCORING KAGGLE AND P2P LENDING US CLUB DATASETS

Dataset	Credit Scoring Kaggle				P2P US Lending Club			
Algorithm	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
LR	72.9	0.7764	0.8661	0.8188	88.9	0.882	0.9948	0.932
LDA	73.69	0.7863	0.8621	0.8224	88.83	0.8779	1	0.935
KNN	74.7	0.8046	0.8479	0.8257	83.39	0.8472	0.9677	0.9034
DT	82.51	0.8833	0.8671	0.8751	83	0.8991	0.8878	0.8934
NB	37.78	0.9019	0.1298	0.2278	85.97	0.8988	0.93	0.9141
RF	85.09	0.8882	0.9026	0.8954	88.9	0.8825	0.9947	0.9323
SVM	73.91	0.7792	0.8803	0.8267	-	-	-	-
AdaBoost	76.99	0.8302	0.8479	0.8389	88.88	0.8807	0.9964	0.935
Gradient Boosting	80.36	0.8574	0.8661	0.8618	88.89	0.879	0.9991	0.9352
Neural Network	77.2	0.8262	0.858	0.8418	88.4	0.8897	0.9765	0.9311
XGB	85.73	0.9275	0.8955	0.8987	88.92	0.8858	0.9895	0.9358

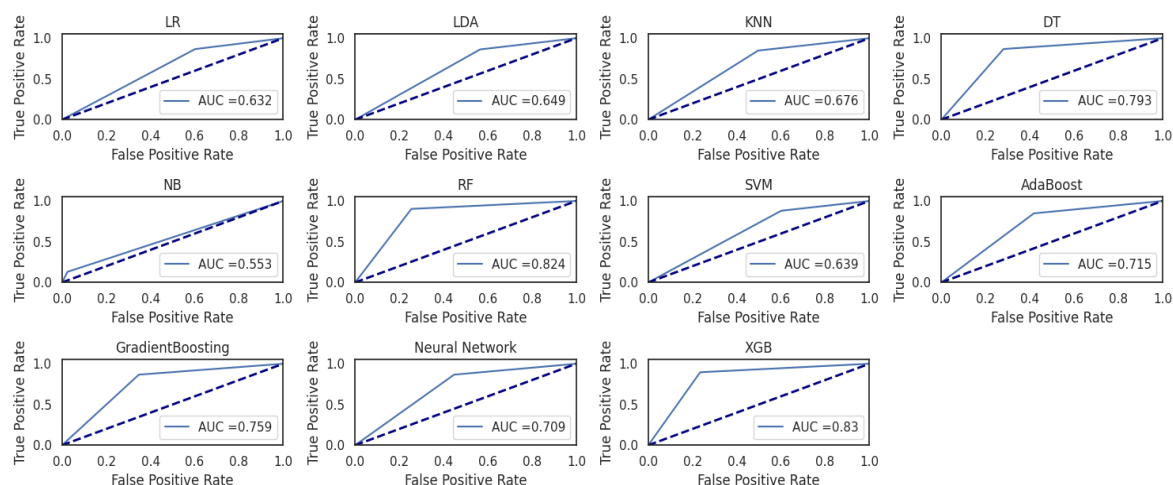


Figure 4. ROC-AUC Curve of Classification Algorithm on Credit Score Kaggle dataset

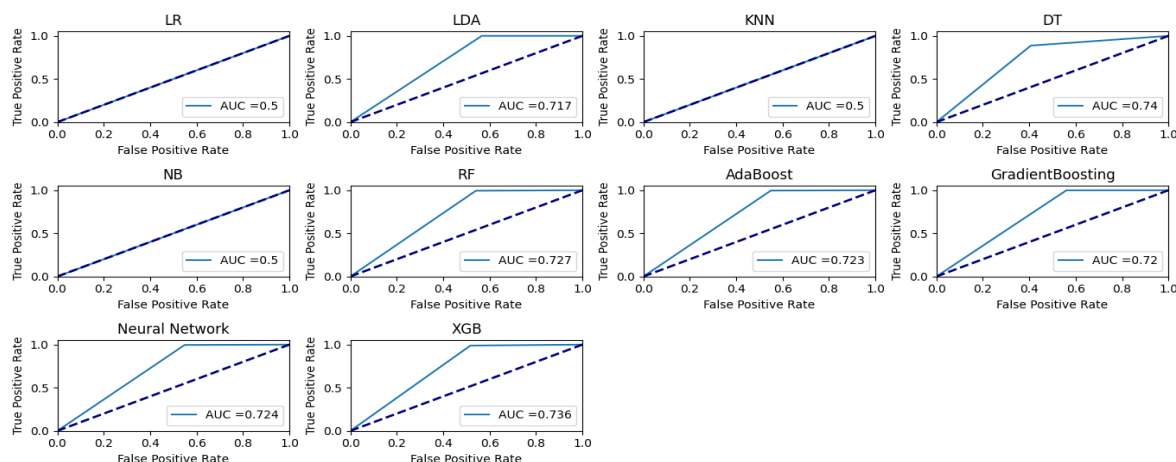


Figure 5. ROC-AUC Curve of Classification Algorithm on P2P US Lending Club dataset

TABLE V
THE PERFORMANCE COMPARISON OF VARIOUS OPTIMIZATION ALGORITHMS ON THE XGBOOST MODEL

Dataset	Model	Accuracy (%)	Precision	Recall	F1
UCI German	XGB	74	0.5758	0.479	0.5229
	GS-XGB	73.75	0.6207	0.3025	0.4068
	RS-XGB	74	0.623	0.3193	0.4222
	BO-XGB	73.5	0.5915	0.3529	0.4421
	Optuna-XGB	75.43	0.6519	0.3445	0.4659
	PSO-XGB	75.25	0.6316	0.4034	0.4923
	HSO- XGB	75.59	0.7466	0.6816	0.6091
	ECS- XGB	74.33	0.7273	0.4068	0.5217
	ISSA HDPM- XGB	75.50	0.6875	0.5593	0.6168
	iCSPM- XGB	75.50	0.7250	0.4915	0.5859
	iCSPM2- XGB	74.56	0.6739	0.5254	0.5905
	WCAPSO-XGB	76.75	0.7879	0.6407	0.6652
UCI Taiwan	XGB	81.49	0.8408	0.9419	0.8884
	GS-XGB	82.36	0.8412	0.9421	0.8952
	RS-XGB	82.24	0.8416	0.9523	0.8935
	BO-XGB	82.5	0.8425	0.9548	0.8952
	Optuna-XGB	82.55	0.8418	0.9568	0.8956
	PSO-XGB	82.57	0.8431	0.9548	0.8955
	HSO- XGB	82.30	0.8427	0.9509	0.8935
	ECS- XGB	82.25	0.8414	0.9522	0.8934
	ISSA HDPM- XGB	82.35	0.8438	0.9499	0.8937
	iCSPM- XGB	82.33	0.8438	0.9496	0.8936
	iCSPM2- XGB	82.28	0.8446	0.9475	0.8931
	WCAPSO-XGB	82.43	0.8431	0.9524	0.8944
P2P US Lending Club	XGB	88.92	0.8858	0.9895	0.9348
	GS-XGB	88.97	0.8852	0.9835	0.9323
	RS-XGB	88.96	0.8860	0.9829	0.9317
	BO-XGB	89.17	0.8853	0.9880	0.9321
	Optuna-XGB	89.35	0.8869	0.9872	0.9338
	PSO-XGB	88.48	0.8862	0.9866	0.9331
	HSO- XGB	88.96	0.8854	0.9905	0.9350
	ECS- XGB	88.96	0.8834	0.9934	0.9352
	ISSA HDPM- XGB	88.96	0.8847	0.9916	0.9351
	iCSPM- XGB	88.96	0.8847	0.9917	0.9351
	iCSPM2- XGB	88.95	0.8840	0.9925	0.9351
	WCAPSO-XGB	88.98	0.8846	0.9921	0.9353
Credit Scoring Kaggle	XGB	85.73	0.9019	0.8955	0.8987
	RS-XGB	85.81	0.898	0.9016	0.8998
	BO-XGB	86.31	0.9027	0.9037	0.9032
	Optuna-XGB	86.27	0.9032	0.9087	0.9060
	PSO-XGB	86.16	0.9001	0.9047	0.9024
	HSO- XGB	86.38	0.8988	0.9097	0.9042
	ECS- XGB	85.95	0.8903	0.9138	0.9019
	ISSA HDPM- XGB	86.24	0.9010	0.9047	0.9028
	iCSPM- XGB	86.38	0.8956	0.9138	0.9046
	iCSPM2- XGB	85.81	0.8956	0.9047	0.9001
	WCAPSO-XGB	88.74	0.9041	0.9087	0.9064

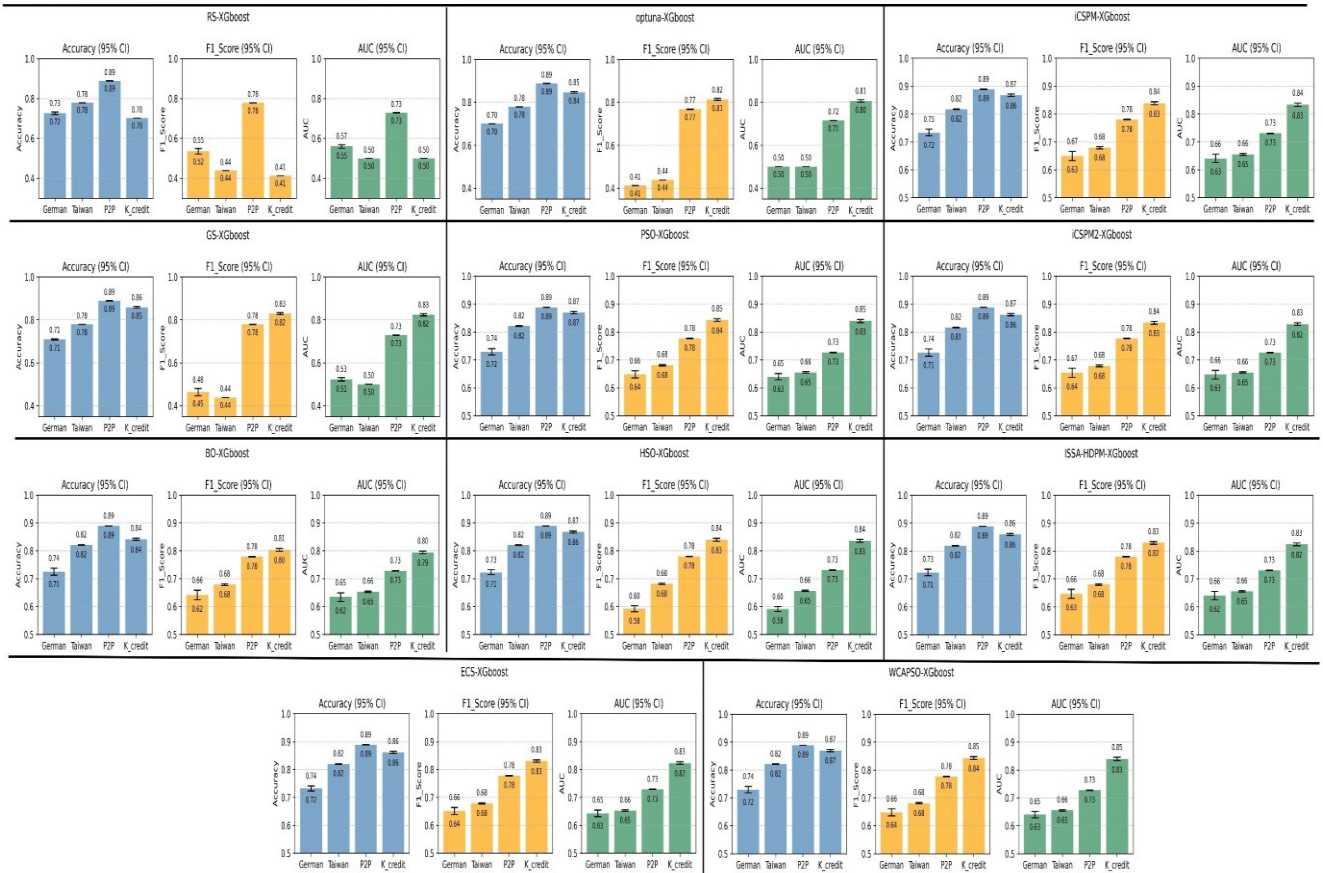


Figure 6. Confidence interval of accuracy, F1-score, and AUC score obtained using various hyperparameter optimization models of XGBoost on four different datasets

Overall, the XGB model performs better on average on all the datasets. XGB has better accuracy in prediction by combining multiple weak tree classifiers. XGB being able to score high in accuracy and F1 score evaluation metrics on the test datasets and covering maximum area in the ROC-AUC curve indicates the XGB model has better classification capability. In credit scoring, all the performance measures play an important role in decision-making. The F1-score of XGB is also high, which indicates the model has very little effect on the imbalanced dataset.

In Table V, the proposed WCAPSO-XGB hyperparameter optimization algorithms were compared to various optimization algorithms, such as grid search (GS), random search (RS), Bays optimization (BO), Optuna optimization, Hybrid Snake Optimizer algorithm (HSO), Exploratory Cuckoo Search (ECS), island Cuckoo Search (iCSPM and iCSPM2), and Improved SSA with HDPM (ISSA HDPM) on four different datasets with varying numbers of instances from small to large. It has been observed that most optimization models, along with WCAPSO-XGB, perform better when handling a large number of features and instances in the dataset.

The experimental results (Table V) show that the proposed model secures better accuracy of 76.75%, 82.43%, 88.98%, and 88.74%; F1-score of 66.52%, 89.44%, 93.53%, and 90.64%; precision of 78.79%, 84.31%, 88.46%, and 90.41%; and recall of 64.07%, 95.24%, 99.21%, and 90.87% in the UCI German, UCI Taiwan dataset, P2P US lending club, and Credit scoring Kaggle dataset, respectively, in comparison with other optimization algorithms. The GS-XGB model was not evaluated on the Credit Scoring Kaggle dataset because of the execution time to evaluate performance matrices. In the UCI German dataset, the performance of all the models suffers due to the lower number of instances in the dataset, which degraded the classification results in terms of various performance metrics. The F1-scores of all models indicate that the dataset is highly unbalanced.

The confidence intervals of the various hyperparameter optimization algorithms on the XGBoost model were evaluated in terms of accuracy, F1 score, and AUC. The confidence interval helps to identify the stability and reliability of the optimized model by evaluating the range within which the actual performance metric lies. To compute the confidence interval of each algorithm, the datasets were split in different ratios, the random state of the split set was 42 to 62, and

stratification was used to balance the distribution of class labels in both the training set and test set during 20 different runs over each XGBoost hyperparameter optimization model on each dataset. The narrow confidence interval indicates that the model exhibits consistent performance over different test samples with low variance. The wide confidence interval indicates that the model performance varied differently for different test samples, indicating model instability and overfitting. The wider confidence interval also indicates more training data for the model. In the experiment, a 95% confidence level was considered, which is suitable for most machine learning algorithms with a 5% error.

In Fig. 6, the proposed WCAPSO-XGB algorithm obtains the confidence interval in accuracy of 72 to 74, 82 to 82, 89 to 89, and 87 to 87; an F1-score of 64 to 66, 68 to 68, 78 to 78, and 84 to 85; and an AUC of 63 to 65, 65 to 66, 73 to 73, and 83 to 85 on UCI German, UCI Taiwan, P2P US Lending Club, and Credit Scoring Kaggle, respectively. In the UCI German dataset (Fig. 6), the accuracy confidence interval of Optuna-XGB had a 0% gap with a mean accuracy of 70%; WCAPSO has a 2.3% gap with a mean accuracy of 72.9%, and the highest gap of 2.5% was in iCSPM2, with a mean accuracy of 72.6%. The F1-score confidence interval of Optuna-XGB had a 0% gap with a mean F1-score of 0.411, while WCAPSO-XGB had a 2.5% gap and a mean F1-score of 0.648. The AUC score confidence interval of Optuna-XGB had a 0% gap with a mean AUC score of 50%, while WCAPSO had a 2.2% gap and a mean AUC score of 63.97%. The results demonstrate that the proposed model performed better with respect to all the confidence intervals, obtaining the highest score with a narrow gap in the UCI German dataset.

In the Credit Scoring Kaggle dataset (Fig. 6), the accuracy confidence interval of RS-XGB had a 0% gap with a mean accuracy of 70.17%, WCAPSO had a 0.79% gap with a mean accuracy of 87%, and the highest gap of 0.85% was in iCSPM with a mean accuracy of 86.67%. The F1-score confidence interval of RS-XGB had a 0% gap with a mean F1-score of 0.412; WCAPSO had a 1% gap with a mean F1-score of 0.842, and ECS had a second minimum of 0.9% gap with a mean F1-score of 0.838. The AUC score confidence interval of RS-XGB had a 0% gap with a mean AUC score of 50%; WCAPSO had a 1.17% gap with a mean AUC score of 83.94%. The results demonstrate that the proposed model performed better with respect to all the confidence intervals, securing the highest score with a narrow gap in the Credit Scoring Kaggle dataset.

In the P2P US Lending Club dataset (Fig. 6), the accuracy confidence interval of WCAPSO-XGB has a 0.078% gap with a mean accuracy of 88.85%, having an almost similar confidence interval gap of 0.076% to 0.084% and 88.8 % with respect to most of the optimization models. The F1-score confidence interval of WCAPSO-XGB has a 0.2% gap with a mean F1-score of 0.777. The AUC score confidence interval of WCAPSO-XGB has a 0.2 % gap with a mean AUC score

of 72.69%. The result shows that with a large number of instances in the dataset (Table I), the proposed model performed almost similarly to other compared optimization models with negligible difference.

In the UCI Taiwan dataset (Fig. 6), the accuracy confidence interval of WCAPSO-XGB has a narrow gap of 0.27% with a mean accuracy of 82%, similar to BO-XGB and PSO-XGB. The F1-score confidence interval of WCAPSO-XGB has a 0.5% gap with a mean F1-score of 0.68, far better than the RS-XGB and Optuna-XGB, which have 0% confidence intervals. The AUC score confidence interval of WCAPSO-XGB has a 0.46 % gap with a mean AUC score of 65.51%, similar to most of the compared optimization algorithms.

Overall, on four datasets, the proposed model has an average confidence interval ranging from 82% to 83% in accuracy, 0.731 to 0.742 in F1-score, and 71% to 72% in AUC, with a confidence of 95% measured over 20 evaluations on each dataset. This shows the proposed model has a narrow gap in confidence interval with a better mean accuracy, F1 score, and AUC score, maintaining a consistent result without overfitting the dataset for the model.

VI. CONCLUSION

Classification of credit score data as good or bad is one of the major tasks for financial companies and banks. Based on the correct classification, financial companies and banks grant loans to potential borrowers, enhancing the revenue of the financial industry. And with misclassification, it leads to huge losses in the industries. For which customer information, such as financial and personal information, must be analyzed carefully to grant or reject the loan application. In this paper, a new variant of PSO has been proposed, named weighted cognitive avoidance particle swarm optimization for the XGBoost model (WCAPSO-XGB), to fine-tune the hyperparameters of the XGBoost model for the credit scoring classification. It has been observed that the XGBoost model performs better as it combines multiple weak learners and forms strong learners. The proposed algorithms perform well for the performance parameters, such as accuracy of 76.75%, 82.43%, 88.98%, and 88.74%; F1-score of 66.52%, 89.44%, 93.53%, and 90.64%; precision of 78.79%, 84.31%, 88.46%, and 90.41%; and recall of 64.07%, 95.24%, 99.21%, and 90.87% in the UCI German, UCI Taiwan dataset, P2P US lending club, and Credit scoring Kaggle dataset, respectively, over the compared algorithms. Overall, the proposed model's accuracy, F1-score, and AUC range from 82% to 83%, 73.1% to 74.2%, and 71% to 72%, respectively, on four datasets. This shows the proposed model has a narrow gap in confidence interval with a better mean accuracy, F1 score, and AUC score, maintaining a consistent result without overfitting the dataset for the model. It provides a better balance between exploration and exploitation, efficient global search, and

parallelization through multiple particle evaluation solutions in parallel.

Further, the performance of the XGBoost model can be enhanced with the optimal tuning of its hyperparameters through WCAPSO parameter settings, including the population size and weighting factors in the update equations (inertia weights (ω)). The limitation of this paper lies with the tuning of WCAPSO inertia weights (ω) which is set randomly between the range of 0.1 to 0.9. The randomness in inertia weights (ω) influences the velocity of the particle and may lead to variation in convergence over the search space. In the future, the proposed model can be further improved with better initialization of inertia weights (ω) and updates during each iteration, feature selection, and parallel boosting and bagging ensemble techniques.

VII. AUTHOR CONTRIBUTIONS

Atul Vikas Lakra: Conceptualization, methodology, formal analysis, investigation, validation, and draft preparation.

Sudarson Jena: Writing—reviewing, editing, and overall supervision.

Kaushik Mishra: Writing—reviewing, editing, and overall supervision.

All authors have read and agreed to the published version of the manuscript.

REFERENCES

- [1] W. Liu, H. Fan, and M. Xia, "Credit scoring based on tree-enhanced gradient boosting decision trees," *Expert Syst Appl*, vol. 189, p. 116034, Mar. 2022, doi: 10.1016/J.ESWA.2021.116034.
- [2] L.-J. C. Kao Chih-Chou; Chiu Fon-Yu, "A Bayesian latent variable model with classification and regression tree approach for behavior and credit scoring," *Knowl Based Syst*, vol. 36, no. NA, pp. 245–252, 2012, doi: 10.1016/j.knsys.2012.07.004.
- [3] Y. Z. Xia Junhao; He Lingyun; Li Yinguo; Niu Mengyi, "A novel tree-based dynamic heterogeneous ensemble method for credit scoring," *Expert Syst Appl*, vol. 159, no. NA, pp. 113615-NA, 2020, doi: 10.1016/j.eswa.2020.113615.
- [4] N.-C. H. Hsieh Lun-Ping, "A data-driven ensemble classifier for credit scoring analysis," *Expert Syst Appl*, vol. 37, no. 1, pp. 534–545, 2010, doi: 10.1016/j.eswa.2009.05.059.
- [5] H. Xiao, Z. Xiao, and Y. Wang, "Ensemble classification based on supervised clustering for credit scoring," *Applied Soft Computing Journal*, vol. 43, 2016, doi: 10.1016/j.asoc.2016.02.022.
- [6] Y. L. Xia Chuanzhe; Da Bowen; Xie Fangming, "A novel heterogeneous ensemble credit scoring model based on bstacking approach," *Expert Syst Appl*, vol. 93, no. NA, pp. 182–199, 2018, doi: 10.1016/j.eswa.2017.10.022.
- [7] A. B. T. Hens Manoj Kumar, "Computational time reduction for credit scoring: An integrated approach based on support vector machine and stratified sampling method," *Expert Syst Appl*, vol. 39, no. 8, pp. 6774–6781, 2012, doi: 10.1016/j.eswa.2011.12.057.
- [8] J. Y. Luo Xin; Tian Ye, "Unsupervised quadratic surface support vector machine with application to credit risk assessment," *Eur J Oper Res*, vol. 280, no. 3, pp. 1008–1017, 2020, doi: 10.1016/j.ejor.2019.08.010.
- [9] C.-L. H. Chuang Szu-Teng, "A hybrid neural network approach for credit scoring," *Expert Syst*, vol. 28, no. 2, pp. 185–196, 2010, doi: 10.1111/j.1468-0394.2010.00565.x.
- [10] P. Zhang, Y. Jia, and Y. Shang, "Research and application of XGBoost in imbalanced data," *Int J Distrib Sens Netw*, vol. 18, no. 6, 2022, doi: 10.1177/15501329221106935.
- [11] C. Qin, Y. Zhang, F. Bao, C. Zhang, P. Liu, and P. Liu, "XGBoost optimized by adaptive particle swarm optimization for credit scoring," *Math Probl Eng*, vol. 2021, 2021, doi: 10.1155/2021/6655510.
- [12] Y.-C. C. Chang Kuei-Hu; Wu Guan Jih, "Application of eXtreme gradient boosting trees in the construction of credit risk assessment models for financial institutions," *Appl Soft Comput*, vol. 73, no. NA, pp. 914–920, 2018, doi: 10.1016/j.asoc.2018.09.029.
- [13] Y. L. Xia Chuanzhe; Liu Nana, "Cost-sensitive boosted tree for loan evaluation in peer-to-peer lending," *Electron Commer Res Appl*, vol. 24, no. NA, pp. 30–49, 2017, doi: 10.1016/j.elerap.2017.06.004.
- [14] J. B. Bergstra Yoshua, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, no. 1, pp. 281–305, 2012, doi: NA.
- [15] S. B. Lessmann Bart; Seow Hsin-Vonn; Thomas Lyn C., "Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research," *Eur J Oper Res*, vol. 247, no. 1, pp. 124–136, 2015, doi: 10.1016/j.ejor.2015.05.030.
- [16] X. C. Dastile Turgay; Potsane Moshe, "Statistical and machine learning models in credit scoring: A systematic literature survey," *Appl Soft Comput*, vol. 91, no. NA, pp. 106263-NA, 2020, doi: 10.1016/j.asoc.2020.106263.
- [17] F. Z. Shen Xingchao; Li Zhiyong; Li Ke; Meng Zhiyi, "A novel ensemble classification model based on neural networks and a classifier optimisation technique for imbalanced credit risk evaluation," *Physica A: Statistical Mechanics and its Applications*, vol. 526, no. NA, pp. 121073-NA, 2019, doi: 10.1016/j.physa.2019.121073.
- [18] S. O. Oreški Goran, "Genetic algorithm-based heuristic for feature selection in credit risk assessment," *Expert Syst Appl*, vol. 41, no. 4, pp. 2052–2064, 2014, doi: 10.1016/j.eswa.2013.09.004.
- [19] S. B. Maldonado Cristián; López Julio; Pérez Juan Eduardo, "Integrated framework for profit-based feature selection and SVM classification in credit scoring," *Decis Support Syst*, vol. 104, no. NA, pp. 113–121, 2017, doi: 10.1016/j.dss.2017.10.007.
- [20] S. Z. Cai Jing, "Exploration of credit risk of P2P platform based on data mining technology," *J Comput Appl Math*, vol. 372, no. NA, pp. 112718-NA, 2020, doi: 10.1016/j.cam.2020.112718.
- [21] C. X. Wang Ran; Qiu Jianfeng; Zhang Xingyi, "AdaBoost-inspired multi-operator ensemble strategy for multi-objective evolutionary algorithms," *Neurocomputing*, vol. 384, no. NA, pp. 243–255, 2020, doi: 10.1016/j.neucom.2019.12.048.
- [22] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.
- [23] J. Mushava and M. Murray, "A novel XGBoost extension for credit scoring class-imbalanced data combining a generalized extreme value link and a modified focal loss function," *Expert Syst Appl*, vol. 202, p. 117233, Sep. 2022, doi: 10.1016/J.ESWA.2022.117233.
- [24] X. Li et al., "Probabilistic solar irradiance forecasting based on XGBoost," *Energy Reports*, vol. 8, 2022, doi: 10.1016/j.egy.2022.02.251.
- [25] J. Mushava and M. Murray, "A novel XGBoost extension for credit scoring class-imbalanced data combining a generalized extreme value link and a modified focal loss function," *Expert Syst Appl*, vol. 202, p. 117233, Sep. 2022, doi: 10.1016/J.ESWA.2022.117233.
- [26] J. Y. Oh, D. H. Ham, Y. G. Lee, and G. Kim, "Short-term Load Forecasting Using XGBoost and the Analysis of Hyperparameters," *Transactions of the Korean Institute of Electrical Engineers*, vol. 68, no. 9, 2019, doi: 10.5370/KIEE.2019.68.9.1073.
- [27] W. Hu, "A Simpler and More Effective Particle Swarm Optimization Algorithm," *Journal of Software*, vol. 18, no. 4, pp. 861-NA, 2007, doi: 10.1360/jos180861.
- [28] A. Biswas, A. V. Lakra, S. Kumar, and A. Singh, "An improved random inertia weighted particle swarm optimization," in *Proceedings - 2013 International Symposium on Computational and Business Intelligence, ISCBI 2013*, IEEE Computer Society, 2013, pp. 96–99, doi: 10.1109/ISCBI.2013.27.
- [29] A. V. Lakra and S. Jena, "Optimization of Random Forest Hyperparameter Using Improved PSO for Handwritten Digits

- Classification,” in *Computing, Communication and Learning: First International Conference, CoCoLe 2022, Warangal, India, October 27–29, 2022, Proceedings*, 2023, pp. 266–276.
- [30] [Dataset] UCI Taiwan “<http://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients>” DOI: 10.24432/C55S3H
- [31] [Dataset] UCI Germany “<http://archive.ics.uci.edu/ml/datasets/Statlog+%28German+Credit+Data%29>”. DOI:10.24432/C5NC77
- [32] [Dataset] Credit Score Kaggle “<https://www.kaggle.com/datasets/parisrohan/credit-score-classification>”
- [33] [Dataset] P2P US Lending Club <https://www.kaggle.com/datasets/jeandiedieunyandwi/lending-club-dataset>
- [34] Alawad, N.A., Abed-alguni, B.H. & El-ibini, M. Hybrid Snake Optimizer Algorithm for Solving Economic Load Dispatch Problem with Valve Point Effect. *J Supercomput* 80, 19274–19323 (2024). <https://doi.org/10.1007/s11227-024-06207-5>
- [35] D. Xue, S. Y. Pang, N. Liu, S. K. Liu, and W. M. Zheng, “Phase-Angle-Encoded Snake Optimization Algorithm for K-Means Clustering,” *Electronics* 2024, Vol. 13, Page 4215, vol. 13, no. 21, p. 4215, Oct. 2024, doi: 10.3390/ELECTRONICS13214215.
- [36] N. A. Alawad, B. H. Abed-alguni, and M. El-ibini, “Hybrid Snake Optimizer Algorithm for Solving Economic Load Dispatch Problem with Valve Point Effect,” *Journal of Supercomputing*, vol. 80, no. 13, pp. 19274–19323, Sep. 2024, doi: 10.1007/S11227-024-06207-5/METRCS.
- [37] B. Li and M. Khayatnezhad, “Modified artificial neural network based on developed snake optimization algorithm for short-term price prediction,” *Heliyon*, vol. 10, no. 5, p. e26335, Mar. 2024, doi: 10.1016/J.HELIYON.2024.E26335.
- [38] E. Shahamatnia and M. M. Ebadzadeh, “Application of particle swarm optimization and snake model hybrid on medical imaging,” *IEEE SSCI 2011 - Symposium Series on Computational Intelligence - CIMI 2011: 2011 IEEE 3rd International Workshop on Computational Intelligence in Medical Imaging*, pp. 23–30, 2011, doi: 10.1109/CIMI.2011.5952043.
- [39] X. -S. Yang and Suash Deb, “Cuckoo Search via Lévy flights,” *2009 World Congress on Nature & Biologically Inspired Computing (NaBIC)*, Coimbatore, India, 2009, pp. 210–214, doi: 10.1109/NABIC.2009.5393690.
- [40] S. Marso and M. El Merouani, “Predicting financial distress using hybrid feedforward neural network with cuckoo search algorithm,” *Procedia Comput Sci*, vol. 170, pp. 1134–1140, Jan. 2020, doi: 10.1016/J.PROCS.2020.03.054.
- [41] H. M. Abdulwahab, S. Ajitha, M. A. N. Saif, B. A. H. Murshed, and F. A. Ghanem, “MOBCSA: Multi-Objective Binary Cuckoo Search Algorithm for Features Selection in Bioinformatics,” *IEEE Access*, vol. 12, pp. 21840–21867, 2024, doi: 10.1109/ACCESS.2024.3362228.
- [42] B. H. Abed-alguni and D. Paul, “Island-based Cuckoo Search with elite opposition-based learning and multiple mutation methods for solving optimization problems,” *Soft comput*, vol. 26, no. 7, pp. 3293–3312, Apr. 2022, doi: 10.1007/S00500-021-06665-6/TABLES/9.
- [43] B. H. Abed-alguni, D. Paul, and R. Hammad, “Improved Salp swarm algorithm for solving single-objective continuous optimization problems,” *Applied Intelligence*, vol. 52, no. 15, pp. 17217–17236, Dec. 2022, doi: 10.1007/S10489-022-03269-X/TABLES/10.
- [44] Y. Wei, W. Wan, X. You, F. Cheng, and Y. Wang, “Improved Salp Swarm Algorithm for Tool Wear Prediction,” *Electronics* 2023, Vol. 12, Page 769, vol. 12, no. 3, p. 769, Feb. 2023, doi: 10.3390/ELECTRONICS12030769.
- [45] N. Singh, S. B. Singh, and E. H. Houssein, “Hybridizing salp swarm algorithm with particle swarm optimization algorithm for recent optimization functions,” *Evol Intell*, vol. 15, no. 1, pp. 23–56, Mar. 2022, doi: 10.1007/S12065-020-00486-6/TABLES/16.
- [46] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997, doi: 10.1109/4235.585893.



interest is optimization techniques in machine learning and cloud computing.

ATUL VIKAS LAKRA received the M.Tech degree from MNNIT, Allahabad in 2013. He is currently an Assistant Professor with the Department of Computer Science and Engineering (CSE), VSSUT, Burla. He is currently PhD scholar at Department of Computer Science Engineering and Application, SUIT, Sambalpur. He is currently working as Assistant Professor, Department of Information Technology, VSSUT, Burla. His main research



published by Springer, Elsevier, ACM, etc. He is a Senior Member of IEEE, Life Member of Computer Society of India (CSI), Association of Indian Science Congress, ISTE, and Senior Member of the International Association of Computer Science and Information Technology. He is currently working as an Associate Professor and Head, Department of Computer Science Engineering and Application at SUIT, Sambalpur. His areas of interest include Parallel and Distributed Computing, Performance and Reliability analysis of Interconnection Networks, High Dimension data, Machine learning, Soft Computing, and Big Data Analytics.

DR. KAUSHIK MISHRA (Member, IEEE) received the Ph.D. degree from the Veer Surendra Sai University of Technology (VSSUT), Burla, in 2021. He is currently an Associate Professor with the Department of Computer Science and Engineering (CSE), Manipal Institute of Technology, Bengaluru, Manipal Academy of Higher Education, Manipal, India. His research interests include task consolidation, load balancing, privacy protection using security mechanisms for cloud computing, fog computing, and the



Internet of Medical Things. He has bagged two best paper awards in two of the conferences. His research findings are published in many journals of repute, such as IEEE Transactions on Industrial Informatics, IEEE Transactions on Intelligent Transportation Systems, IEEE Transactions on Network and Service Management, ACM Transactions on Internet Technology, Journal of Supercomputing, Journal of King Saud University, and Open Computer Science; and conferences like IEEE TENSYP. He has served as a Reviewer for many reputed journals, such as IEEE Transactions on Intelligent Transportation Systems, IEEE Internet of Things, Future Generation Computing Systems, Cluster Computing, Journal of Supercomputing, Multimedia Tools and Applications, Journal of Grid Computing, and IGI Global.